



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

Trabalho Prático

Autor / Author

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767



INSTITUTO POLITÉCNICO DE
COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, 29 maio de 2025

ÍNDICE

ÍNDICE	2
Índice de Figuras	2
Introdução	3
Estruturas de Dados e Constantes	4
Arquitetura	5
Arbitro	5
Jogouí	5
Comunicação.....	5
Gestão de <i>Threads</i> e acesso aos dados	6
Conclusão	8

Índice de Figuras

Figura 1 – Esquema de funcionamento	3
Figura 2 - Estrutura principal de dados utilizada	4

Introdução

Este relatório descreve o funcionamento, as implementações e os desafios técnicos de um sistema cliente-servidor que utiliza Named Pipes e memória partilhada para a comunicação. O trabalho desenvolvido utilizando a Windows API é um jogo que permite aos jogadores acertar palavras que se encontram num dicionário que é gerido por um arbitro.

O sistema é composto por:

Arbitro: Gestão de recursos e de jogo

Jogoui: Sistema que permite um utilizador jogar

Bot: Sistema que vai jogar contra os utilizadores de forma automática

Named Pipes: Canais de comunicação, entre arbitro e jogoui

Memória partilhada: Utilizada para partilhar dados interprocessos

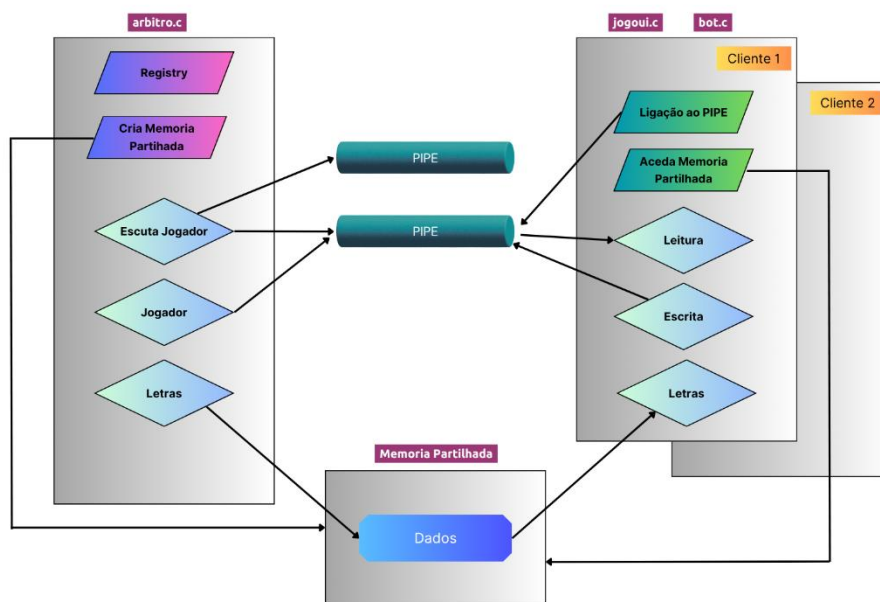


Figura 1 – Esquema de funcionamento

Estruturas de Dados e Constantes

As estruturas de dados, constantes e bibliotecas são definidas no ficheiro *utils.h*. A estrutura principal é composta pelos mecanismos de sincronização e pela estrutura de dados da memória partilhada.

```

typedef struct {
    HKEY chave;
    int max_letras; // MAXLETRAS lido do registry
    int ritmo;      // RITMO lido do registry
} ConfigGame;

typedef struct {
    float pontos;
    TCHAR username[TAM];
    int tipo; //tipo = 1 bot tipo=0 jogoui
    HANDLE hPipe;
} Players;

typedef struct {
    TCHAR letras_visiveis[MAX_LETRAS];
    TCHAR ultima_pal[MAX_LETRAS];
    Players jogadores[MAX_PLAYERS];
    unsigned int num_jogadores;
    int id_best_player;
    int num_letras;
    BOOL jogoIniciado;
} SharedMem;

typedef struct {
    unsigned int shutdown; // flag "continua". 0 = continua, 1 = deve terminar
    HANDLE hEv;
    HANDLE hMapFile;      // ficheiro de memoria
    SharedMem* sharedMem; // memoria partilhada
    HANDLE hMutex;        // mutex - trabalho de casa -> acrescentar os outros 2 mutexes
    HANDLE hWriteSem;     // semáforo "aguarda por itens escritos"
    HANDLE hReadSem;      // semáforo "aguarda por posições vazias"
    HANDLE hPipe;         //Pipe
    ConfigGame cfg;
} Data;

```

Figura 2 - Estrutura principal de dados utilizada

Arquitetura

O sistema foi projetado em três componentes, *arbitro*, *jogoui.c* e *bot.c*

Arbitro

- Inicialização dos recursos partilhadas e dos Named Pipes.
- Gestão de comandos inseridos por parte do administrador
- Gestão de comandos de administrador e de mensagens vindas dos jogadores
- Utilização de *threads* para o correto funcionamento do servidor tendo em conta o exigido.
 - *Thread* EsperaJogador: fica a espera de uma conexão de um jogador
 - *Thread* Jogador: recebe os comandos via Named Pipe de cada jogador e processa o pedido
 - *Thread* Letras: gera letras aleatórias e insere-as na memória partilhada

Jogoui

- Permite a interação do utilizador com o servidor através de comandos e palavras através de Named Pipe.

Comunicação

- Named Pipes utilizado para a comunicação de comandos entre os processos, com comunicação bidirecional e assíncrona com o uso do OverlapIO. O arbitro aguarda uma ligação ao Named Pipe e lança uma thread para cada jogador.
- Memória Partilhada utilizada para o acesso aos recursos do jogo, numero de jogadores ativos, letras disponíveis.

Gestão de *Threads* e acesso aos dados

O processo Jogoui é composto por três threads principais: uma responsável pela leitura do stdin (entrada padrão) para capturar os comandos do utilizador; uma segunda thread dedicada à leitura de mensagens provenientes do Named Pipe; e uma terceira thread dedicada à leitura das letras através da memória partilhada.

O encerramento do processo pode ocorrer de duas formas: por comando direto do utilizador ou de forma forçada pelo administrador, através do arbitro.

A estrutura de dados mantida pelo arbitro regista informações essenciais de cada jogador, incluindo o nome, o identificador do Named Pipe (handle) e a pontuação acumulada durante o jogo.

O processo Bot segue uma lógica semelhante ao Jogoui, com a diferença de que utiliza um conjunto predefinido de palavras para interagir com o jogo e pontuar automaticamente, sem intervenção do utilizador.

O processo arbitro organiza-se em duas partes principais. A função principal (main) é responsável pela leitura de comandos administrativos, pelo lançamento de bots mediante instruções específicas e pela criação das threads que controlam as conexões dos jogadores, uma de escuta de nova ligação que depois lança uma thread para tratar de cada jogador e geram letras aleatórias para o jogo.

ID	Descrição da funcionalidade / requisito	Estado
1	Comunicação entre árbitro e jogadores através de Named Pipes	Implementado
2	Utilização de memória partilhada para partilhar letras visíveis entre árbitro e jogadores	Implementado
3	Uso de eventos e mutex para sincronização no acesso à memória partilhada	Implementado
4	Verificação de palavras através de dicionário externo (ficheiro dicionario.txt)	Implementado
5	Comando :pont devolve pontuação atual ao jogador	Implementado
6	Comando :jogs devolve lista dos jogadores ativos	Implementado
7	Comando :sair permite o jogador sair, notificando o árbitro e removendo-o da memória partilhada	Implementado
8	Mecanismo de letras aleatórias com ritmo controlado via parâmetro RITMO (do registry)	Implementado
9	Leitura de parâmetros MAXLETRAS e RITMO a partir do registry	Implementado
10	Gestão dinâmica de jogadores (entrada e saída), com notificação a todos os jogadores	Implementado
11	Implementação do bot automático com tempo de reação aleatório e lógica baseada em dicionário interno	Implementado

12	Exclusão de jogadores através do comando excluir no árbitro	Implementado
13	Comando listar mostra a lista atual de jogadores e respetivos pontos	Implementado
14	Comando iniciarbot permite lançar bots automaticamente a partir do árbitro	Implementado
15	Comandos acelerar e travar alteram o ritmo de jogo dinamicamente	Implementado
16	Encerramento do jogo através do comando encerrar, notificando todos os jogadores com :fim	Implementado
17	Sincronização e proteção contra nomes duplicados de jogadores	Implementado
18	Atualização da memória partilhada com letras visíveis, substituindo letras usadas e mantendo espaços com _	Implementado
19	Sistema de pontos com incremento por palavra válida e penalização por palavra inválida	Implementado
20	Controlo por parte do árbitro da vida útil das letras através de cadência e uso de temporizador	Implementado
21	Implementação do programa painel com interface gráfica	Não Implementado

Conclusão

O desenvolvimento deste sistema cliente-servidor utilizando a Windows API, com destaque para os mecanismos de comunicação interprocesso como Named Pipes e memória partilhada, permitiu explorar conceitos avançados de concorrência, sincronização e gestão de processos.

A utilização de threads tanto no servidor quanto nos clientes mostrou-se fundamental para garantir a responsividade e paralelismo necessários ao bom funcionamento do sistema. Adicionalmente, a gestão rigorosa dos recursos partilhados e das estruturas de dados assegurou a integridade da informação e a coerência da lógica de jogo.

Este trabalho evidenciou a importância do conhecimento detalhado da Windows API para a implementação de sistemas robustos e eficientes em ambiente Windows.